

Name: Javaughn Stevens

CMSC 204 DictionaryBuilder Project 4 Writeup

1. What did you learn?

I learned about how I can create a simple dictionary using hash tables and linked list sorting that can process separate .txt files as well as the number of estimated entries. I also learned about how a program might store the data that humans interpret as readable text (e.g. words entered into the dictionary have hash codes associated with them).

2. What issues did you encounter, if any?

I ran into issues with testing the DictionaryShell main method because this is something I had never done before (in all previous projects the main method was in the GUI and I was not responsible for personally writing tests for this method).

3. What would you have done differently?

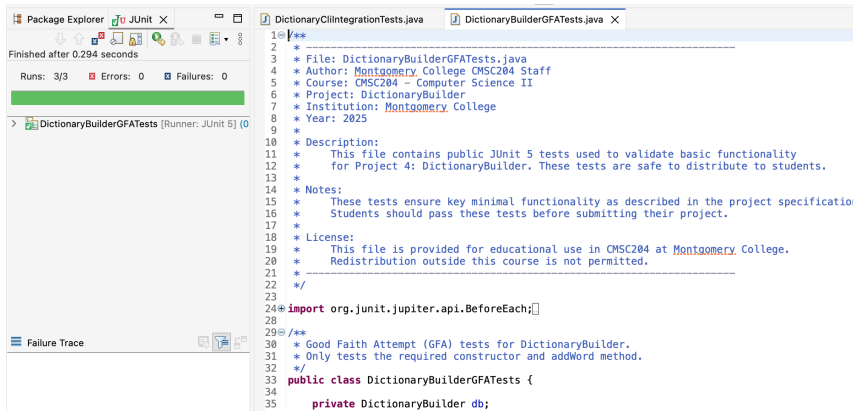
I would spend more time reviewing the video notes from the weekly modules because they seem to focus more on implementing the data structures I need to master and I referred to them multiple times when working on this project implementation.

4. How can you apply this concept in the future?

I can apply this concept in the future because this will potentially allow me to create programs that work with many outside .txt files and hash codes that give each element a unique identity. This can be used for systems of records, statistics, and other large storage systems. I feel that many array and linked implementations have an emphasis on storing large amounts of data in the most efficient manner possible (

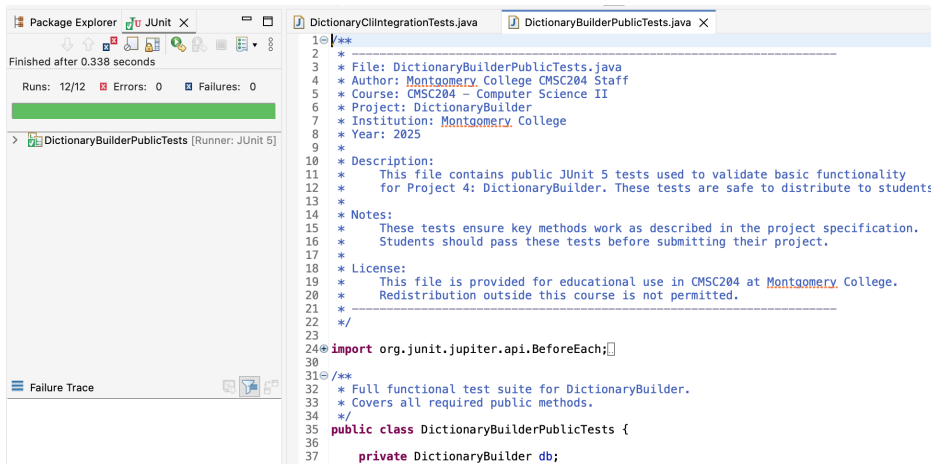
JUnit Test Screenshots:

DictionaryBuilderGFA Tests.java



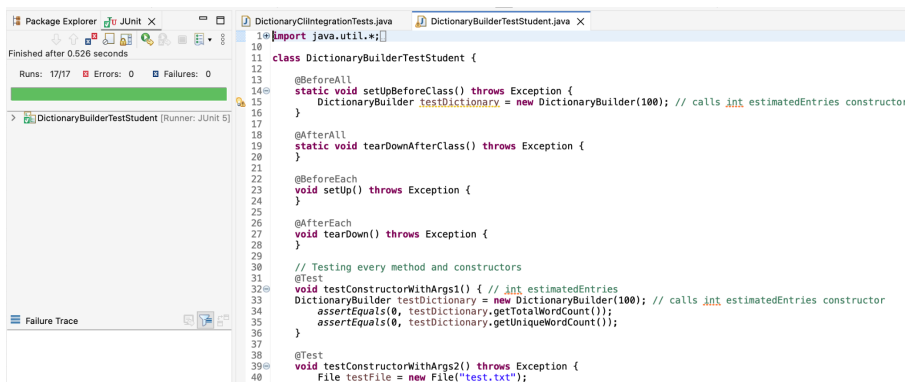
```
1  /**
2   *
3   * File: DictionaryBuilderGFA Tests.java
4   * Author: Montgomery College CMSC204 Staff
5   * Course: CMSC204 - Computer Science II
6   * Project: DictionaryBuilder
7   * Institution: Montgomery College
8   * Year: 2025
9   *
10  * Description:
11  * This file contains public JUnit 5 tests used to validate basic functionality
12  * for Project 4: DictionaryBuilder. These tests are safe to distribute to students.
13  *
14  * Notes:
15  * These tests ensure key minimal functionality as described in the project specification.
16  * Students should pass these tests before submitting their project.
17  *
18  * License:
19  * This file is provided for educational use in CMSC204 at Montgomery College.
20  * Redistribution outside this course is not permitted.
21  */
22
23
24  import org.junit.jupiter.api.BeforeEach;
25
26
27  /**
28   * Good Faith Attempt (GFA) tests for DictionaryBuilder.
29   * Only tests the required constructor and addWord method.
30   */
31
32  public class DictionaryBuilderGFA Tests {
33
34      private DictionaryBuilder db;
35  }
```

DictionaryBuilderPublic Tests.java



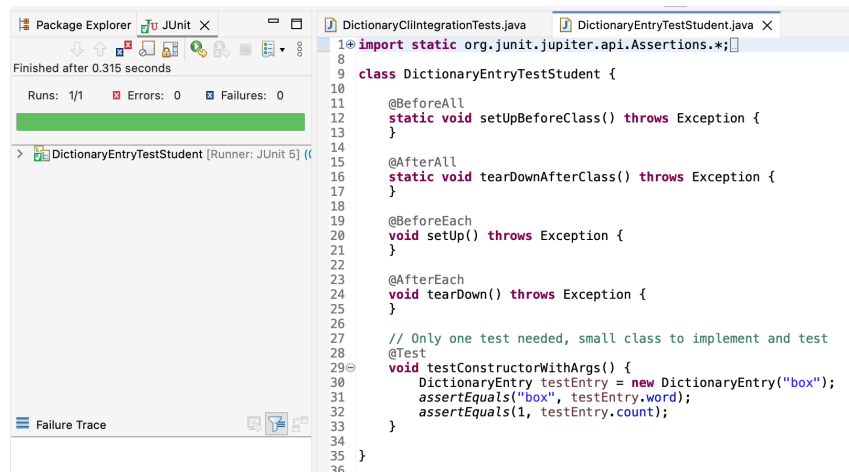
```
1  /**
2   *
3   * File: DictionaryBuilderPublic Tests.java
4   * Author: Montgomery College CMSC204 Staff
5   * Course: CMSC204 - Computer Science II
6   * Project: DictionaryBuilder
7   * Institution: Montgomery College
8   * Year: 2025
9   *
10  * Description:
11  * This file contains public JUnit 5 tests used to validate basic functionality
12  * for Project 4: DictionaryBuilder. These tests are safe to distribute to students.
13  *
14  * Notes:
15  * These tests ensure key methods work as described in the project specification.
16  * Students should pass these tests before submitting their project.
17  *
18  * License:
19  * This file is provided for educational use in CMSC204 at Montgomery College.
20  * Redistribution outside this course is not permitted.
21  */
22
23
24  import org.junit.jupiter.api.BeforeEach;
25
26
27  /**
28   * Full functional test suite for DictionaryBuilder.
29   * Covers all required public methods.
30   */
31
32  public class DictionaryBuilderPublic Tests {
33
34      private DictionaryBuilder db;
35  }
```

DictionaryBuilderTest Student.java



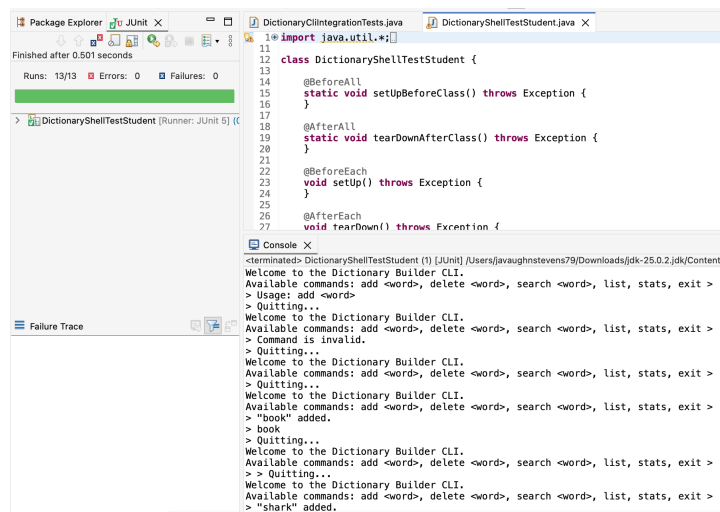
```
1  import java.util.*;
2
3  class DictionaryBuilderTest Student {
4
5      @BeforeAll
6      static void setUpBeforeClass() throws Exception {
7          DictionaryBuilder testDictionary = new DictionaryBuilder(100); // calls int estimatedEntries constructor
8      }
9
10     @AfterAll
11     static void tearDownAfterClass() throws Exception {
12     }
13
14     @BeforeEach
15     void setUp() throws Exception {
16     }
17
18     @AfterEach
19     void tearDown() throws Exception {
20     }
21
22     // Testing every method and constructors
23     @Test
24     void testConstructorWithArgs1() { // int estimatedEntries
25         DictionaryBuilder testDictionary = new DictionaryBuilder(100); // calls int estimatedEntries constructor
26         assertEquals(0, testDictionary.getTotalWordCount());
27         assertEquals(0, testDictionary.getUniqueWordCount());
28     }
29
30     @Test
31     void testConstructorWithArgs2() throws Exception {
32         File testFile = new File("test.txt");
33     }
34 }
```

DictionaryEntryTestStudent.java



```
1 import static org.junit.jupiter.api.Assertions.*;
2
3 class DictionaryEntryTestStudent {
4
5     @BeforeAll
6     static void setUpBeforeClass() throws Exception {
7     }
8
9     @AfterAll
10    static void tearDownAfterClass() throws Exception {
11    }
12
13    @BeforeEach
14    void setUp() throws Exception {
15    }
16
17    @AfterEach
18    void tearDown() throws Exception {
19    }
20
21    // Only one test needed, small class to implement and test
22    @Test
23    void testConstructorWithArgs() {
24        DictionaryEntry testEntry = new DictionaryEntry("box");
25        assertEquals("box", testEntry.word);
26        assertEquals(1, testEntry.count);
27    }
28
29 }
```

DictionaryShellTestStudent.java

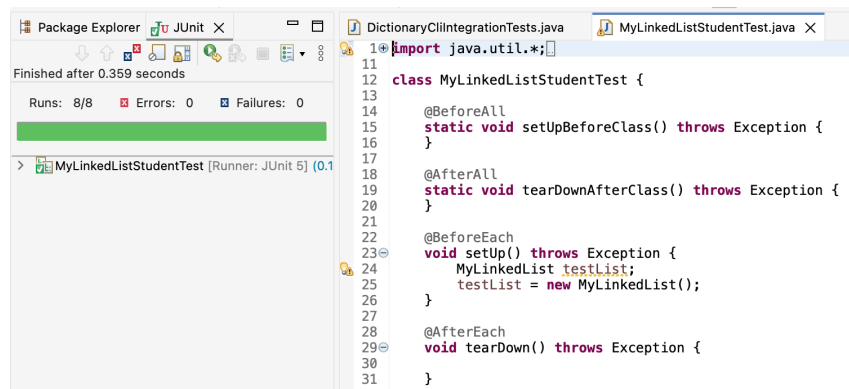


```
1 import java.util.*;
2
3 class DictionaryShellTestStudent {
4
5     @BeforeAll
6     static void setUpBeforeClass() throws Exception {
7     }
8
9     @AfterAll
10    static void tearDownAfterClass() throws Exception {
11    }
12
13    @BeforeEach
14    void setUp() throws Exception {
15    }
16
17    @AfterEach
18    void tearDown() throws Exception {
19    }
20
21 }
```

Console X

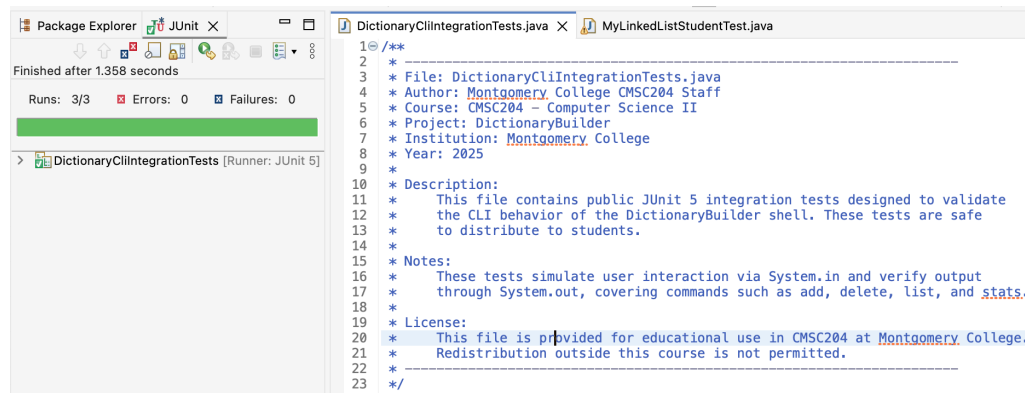
```
<terminated> DictionaryShellTestStudent (1) [JUnit] /Users/javaughnstevens79/Downloads/jdk-25.0.2.jdk/Contents
Welcome to the Dictionary Builder CLI.
Available commands: add <word>, delete <word>, search <word>, list, stats, exit >
> Usage: add <word>
> Quitting...
Welcome to the Dictionary Builder CLI.
Available commands: add <word>, delete <word>, search <word>, list, stats, exit >
> Command is invalid.
> Quitting...
Welcome to the Dictionary Builder CLI.
Available commands: add <word>, delete <word>, search <word>, list, stats, exit >
> Quitting...
Welcome to the Dictionary Builder CLI.
Available commands: add <word>, delete <word>, search <word>, list, stats, exit >
> "book" added.
> book
> Quitting...
Welcome to the Dictionary Builder CLI.
Available commands: add <word>, delete <word>, search <word>, list, stats, exit >
> > Quitting...
Welcome to the Dictionary Builder CLI.
Available commands: add <word>, delete <word>, search <word>, list, stats, exit >
> "shark" added.
> shark
> shark
> Quitting...
```

MyLinkedListStudentTest.java



```
1 import java.util.*;
2
3 class MyLinkedListStudentTest {
4
5     @BeforeAll
6     static void setUpBeforeClass() throws Exception {
7     }
8
9     @AfterAll
10    static void tearDownAfterClass() throws Exception {
11    }
12
13    @BeforeEach
14    void setUp() throws Exception {
15        MyLinkedList testList;
16        testList = new MyLinkedList();
17    }
18
19    @AfterEach
20    void tearDown() throws Exception {
21    }
22
23 }
```

DictionaryCliIntegrationTests.java

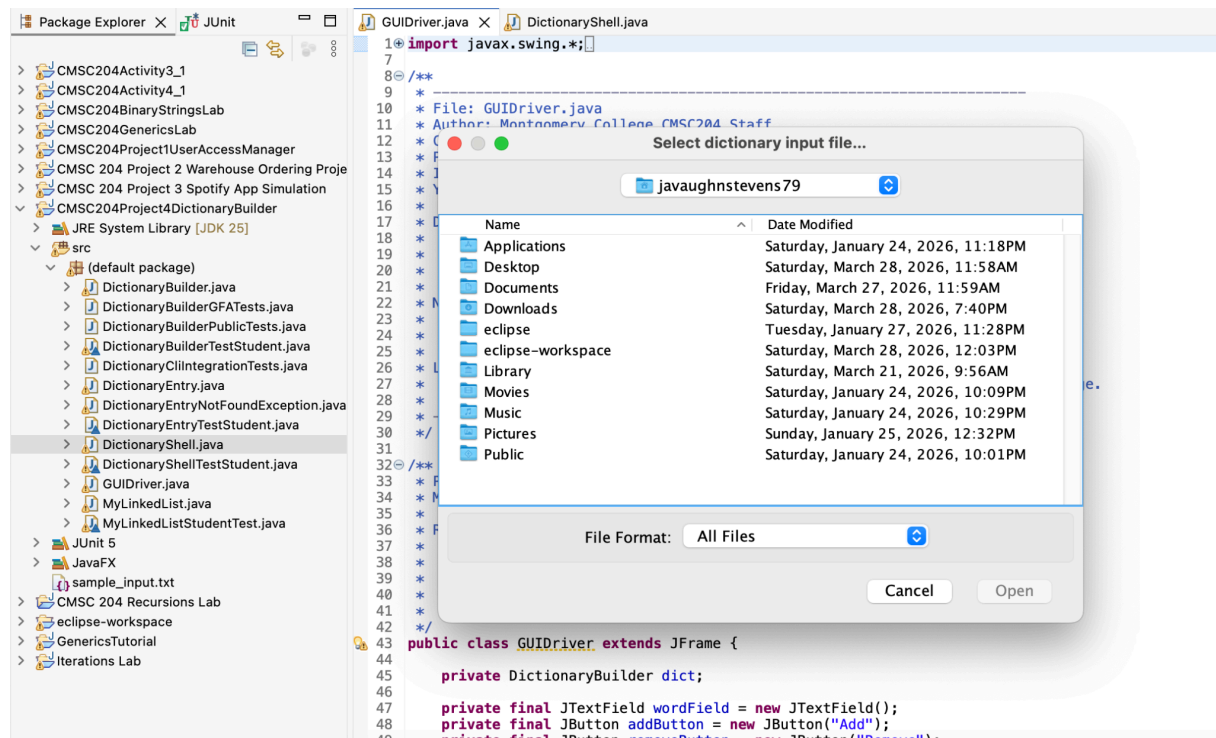


Program Run Screenshots:

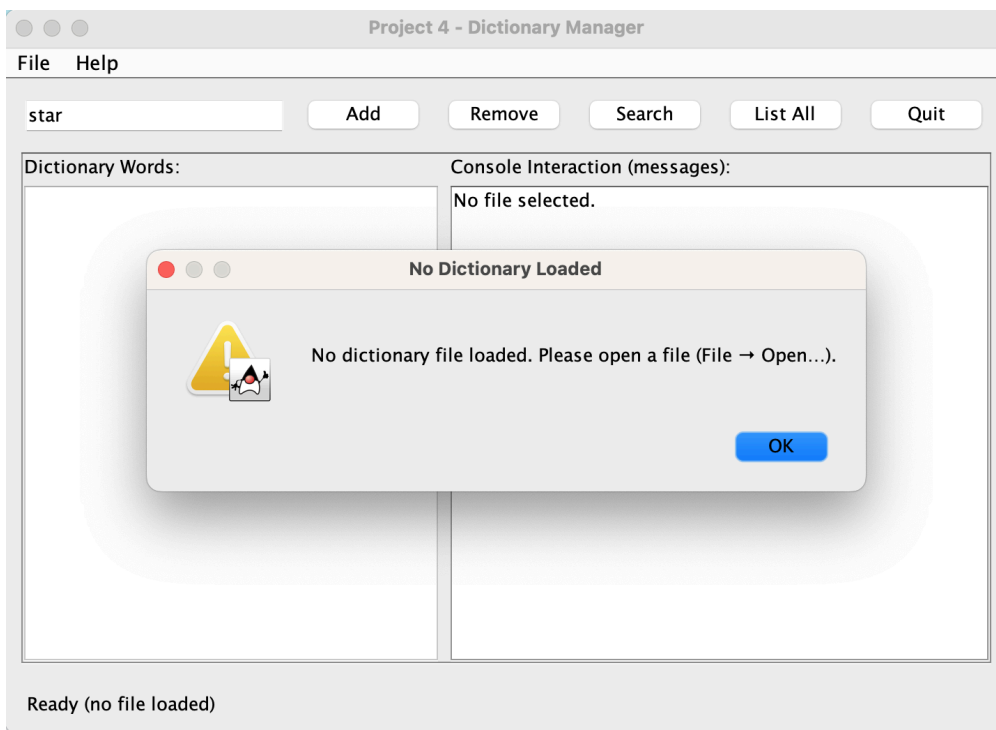
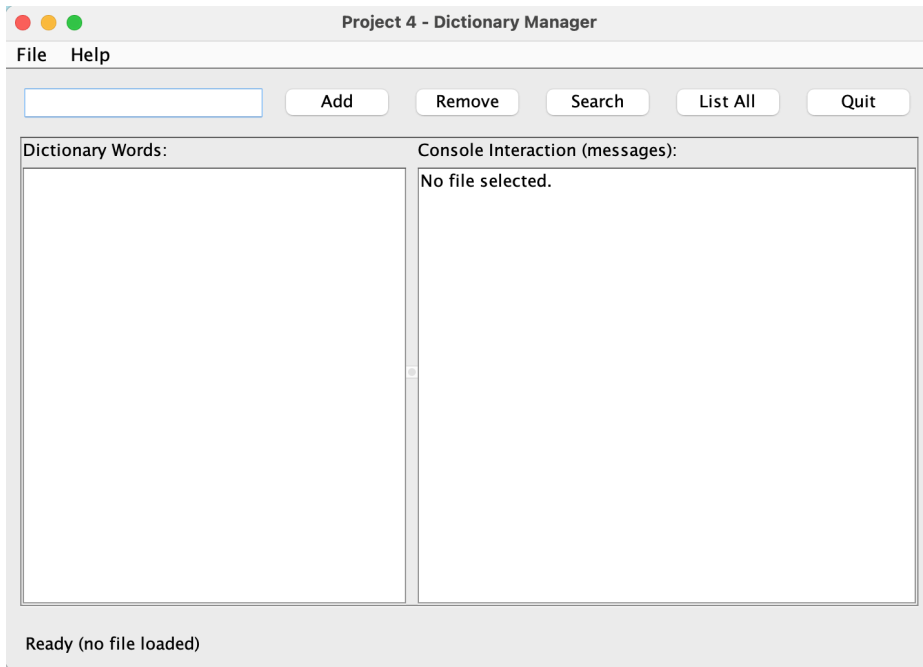
DictionaryShell.java (main method #1)

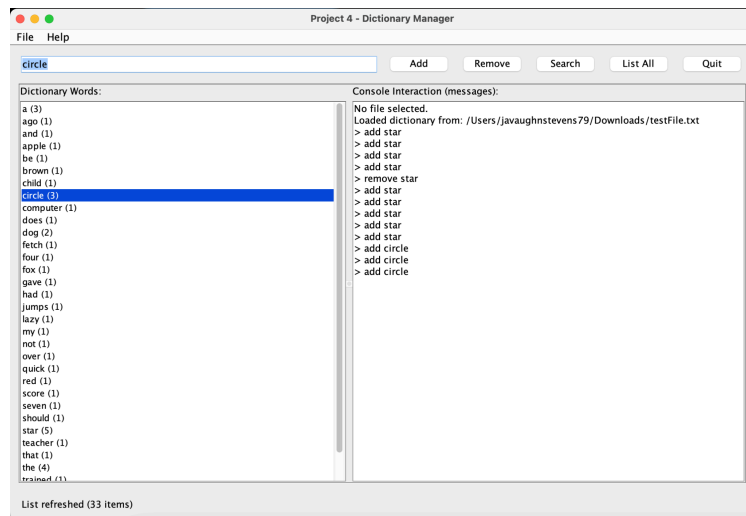
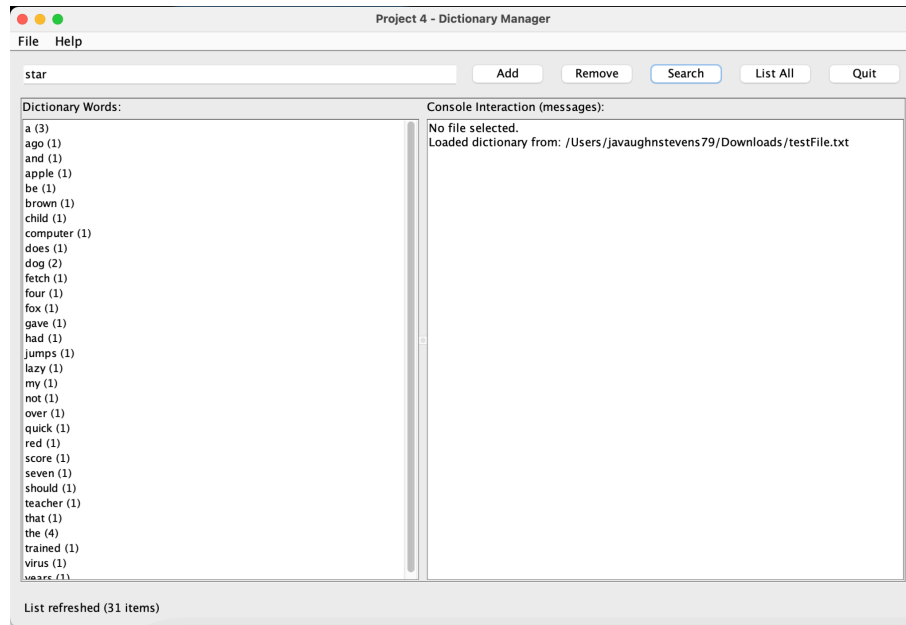
```
Welcome to the Dictionary Builder CLI.
Available commands: add <word>, delete <word>, search <word>, list, stats, exit >
> add fox
"fox" added.
> add fox
"fox" added.
> add fox
"fox" added.
> delete fox
"fox" deleted.
> add rabbit
"rabbit" added.
> search rabbit
1 instance(s) of "rabbit" found.
> search fox
"fox" not found.
> search monkey
"monkey" not found.
> delete monkey
The word monkey was not found.
> lis
Command is invalid.
> list
rabbit
> stats
Total words: 1
Total unique words: 1
Estimated load factor: 0.01
> exit
Quitting...
```

GUIDriver.java (main method #2)



(More on next pages)





Note: I received minor insights about JUnit testing in Java main() methods in the following articles:

1)

<https://stackoverflow.com/questions/36349827/testing-main-method-by-junit>

2)

<https://examples.javacodegeeks.com/test-main-method-with-junit/>